

Preventia360 External API (v1)

Admin Guide — Token Management

Audience: Preventia360 administrators and CS agents who issue, scope and revoke API keys for third-party integrations (bots, CRMs, ops tools). Integrators receive a separate *Integrator Guide* and never need anything in this document except the key itself.

Overview

The Preventia360 External API is a read-only HTTP API that lets third-party applications render Preventia's gig booking forms and pricing outside our own apps — for example, a WhatsApp bot that walks a customer through booking a sahayak visit. Access is controlled by per-integration API keys, which are managed from the Admin Portal using the endpoints in this guide. You do this once per integration, not per request.

Base URL & response envelope

```
https://<your-backend-host>/api/v1
```

(Backend context-path is `/api/v1`; every path below is relative to that.)

Every successful call — on both APIs — returns the same envelope:

```
{
  "success": true,
  "data": { "...": "..."},
  "timestamp": "2026-10-01T12:34:56.789Z"
}
```

Every handled error returns:

```
{
  "success": false,
  "errorCode": "ROLE_OUT_OF_SCOPE",
  "message": "This API key is not scoped for role 'DOCTOR'.",
  "messageKey": "errors.ROLE_OUT_OF_SCOPE",
  "timestamp": "2026-10-01T12:34:56.789Z"
}
```

Exception — authentication failures. A missing, malformed or revoked bearer key (and an admin-portal request with no/expired login) does **not** go through that JSON envelope — it's rejected at the security-filter layer before any controller runs, as a bare **403 Forbidden** with an empty body. Treat any **403** with no JSON body as "the credential wasn't accepted"; a **403** with a JSON body (`errorCode` present) is a business-rule rejection like `ROLE_OUT_OF_SCOPE` instead.

Token management endpoints

Base path: `/admin/external-clients`. Authenticated the normal admin way — an `Authorization: Bearer <admin JWT>` header from an Admin Portal login (not an API key). Also reachable by clicking around **Admin Portal** → **Config** → **External API Access**, which calls these same endpoints.

Action	Method & path	Who
List clients	GET <code>/admin/external-clients</code>	ADMIN, CS_AGENT
Create a client	POST <code>/admin/external-clients</code>	ADMIN only
Change scope	PUT <code>/admin/external-clients/{id}/scopes</code>	ADMIN only
Revoke	PUT <code>/admin/external-clients/{id}/revoke</code>	ADMIN only
Reactivate	PUT <code>/admin/external-clients/{id}/reactivate</code>	ADMIN only
Delete	DELETE <code>/admin/external-clients/{id}</code>	ADMIN only

A client's `id` (UUID) is only used for admin management; your integration never sends it anywhere — only the raw key matters at runtime.

1.1 List clients

```
GET /admin/external-clients
Authorization: Bearer <admin JWT>
```

The raw key is never returned here (or anywhere) after creation — only `keyPrefix`, a short slice kept for telling rows apart.

Response

```
{
  "success": true,
  "data": [
    {
      "id": "0b5b6e0a-3c2e-4b0a-9a3e-1f2d3c4b5a6f",
      "name": "Ops WhatsApp bot",
      "description": "Reads the sahayak catalog for the booking flow",
      "keyPrefix": "pv_live_8f2a91c4",
      "scopes": ["SAHAYAK"],
      "active": true,
      "createdAt": "2026-09-30T09:12:00Z",
      "lastUsedAt": "2026-10-01T11:58:21Z",
      "revokedAt": null
    },
    {
      "id": "1c6c7f1b-4d3f-5c1b-0b4f-2e3e4d5c6b7a",
      "name": "Legacy CRM (decommissioned)",
      "description": null,
      "keyPrefix": "pv_live_a1b2c3d4",
      "scopes": [],
      "active": false,
      "createdAt": "2026-07-14T10:00:00Z",
      "lastUsedAt": "2026-08-01T08:30:00Z",
      "revokedAt": "2026-08-02T00:00:00Z"
    }
  ],
  "timestamp": "2026-10-01T12:00:00Z"
}
```

`scopes: []` means **unrestricted** — that key can query every provider role.

1.2 Create a client

```
POST /admin/external-clients
Authorization: Bearer <admin JWT>
Content-Type: application/json
{
  "name": "Ops WhatsApp bot",
  "description": "Reads the sahayak catalog for the booking flow",
  "scopes": ["SAHAYAK"]
}
```

`name` is required. `description` and `scopes` are optional — omit `scopes` (or send `[]`) for a key that can read every provider role.

Response — the ONLY time the raw key is ever returned:

```
{
  "success": true,
  "message": "Copy this key now — it will not be shown again.",
  "data": {
    "id": "0b5b6e0a-3c2e-4b0a-9a3e-1f2d3c4b5a6f",
    "name": "Ops WhatsApp bot",
    "description": "Reads the sahayak catalog for the booking flow",
    "keyPrefix": "pv_live_8f2a91c4",
    "scopes": ["SAHAYAK"],
    "active": true,
    "createdAt": "2026-10-01T12:00:00Z",
    "lastUsedAt": null,
    "revokedAt": null,
    "apiKey": "pv_live_8f2a91c4k3nQ7z1xR5pL9mW2vY8bT4sU6hC0eF3gD1aN7jK5i"
  },
  "timestamp": "2026-10-01T12:00:00Z"
}
```

Store `apiKey` in your integration's own secrets vault immediately — a second `GET` on this client will never include it again. If it's lost, create a new client and revoke the old one; there is no "regenerate key" action.

1.3 Change scope

```
PUT /admin/external-clients/{id}/scopes
Authorization: Bearer <admin JWT>
Content-Type: application/json
{ "scopes": ["SAHAYAK", "NURSE"] }
```

Send `{ "scopes": [] }` to make the key unrestricted again.

Response — same shape as 1.1's row, with the new `scopes`.

1.4 Revoke

```
PUT /admin/external-clients/{id}/revoke
Authorization: Bearer <admin JWT>
```

No body. Immediately — every call using this key starts getting `403` (empty body, see above). The row is kept (for its history) with `active: false` and `revokedAt` stamped.

Response

```
{
  "success": true,
  "data": {
    "id": "0b5b6e0a-3c2e-4b0a-9a3e-1f2d3c4b5a6f",
    "name": "Ops WhatsApp bot",
    "description": "Reads the sahayak catalog for the booking flow",
    "keyPrefix": "pv_live_8f2a91c4",
    "scopes": ["SAHAYAK"],
    "active": false,
    "createdAt": "2026-10-01T12:00:00Z",
    "lastUsedAt": "2026-10-01T11:58:21Z",
    "revokedAt": "2026-10-01T13:00:00Z"
  },
  "timestamp": "2026-10-01T13:00:00Z"
}
```

1.5 Reactivate

```
PUT /admin/external-clients/{id}/reactivate
Authorization: Bearer <admin JWT>
```

No body. Un-revokes a client **without** issuing a new key — the original raw key (which the integration already has stored) starts working again. Response shape: same as 1.4, with **active: true** and **revokedAt: null**.

1.6 Delete

```
DELETE /admin/external-clients/{id}
Authorization: Bearer <admin JWT>
```

Permanently removes the row. Prefer **Revoke** unless you're cleaning up test data — deleting loses the audit trail (who created it, when it was last used).

Response

```
{ "success": true, "data": null, "timestamp": "2026-10-01T13:05:00Z" }
```

Handing a key to an integrator

- Share the raw **apiKey** once, over a secure channel (password manager share or secrets vault) — never over email or chat.
- Scope the key to the minimum set of provider roles the integration needs; avoid unrestricted (**scopes: []**) keys for third parties.
- Keys have no expiry. Review the client list periodically using **lastUsedAt**, and revoke clients that are no longer in use.
- If a key is lost or suspected leaked, create a new client, hand over the new key, then revoke the old one.

One-time setup — curl example

```
BASE=https://api.preventia360.com/api/v1
# 1. One-time setup – create a client as an admin (or do it from the Admin Portal UI)
ADMIN_JWT=...
curl -s -X POST \
  -H "Authorization: Bearer $ADMIN_JWT" -H "Content-Type: application/json" \
  -d '{"name": "Ops WhatsApp bot", "scopes": ["SAHAYAK"]}' \
  "$BASE/admin/external-clients" | jq
# → copy data.apiKey from the response, store it securely. It is never shown again.
```

Offline paper form (admin-only, separate feature)

For a field worker heading somewhere with no connectivity, the admin portal can generate a printable PDF of a category's entire PRE/DURING/POST form set (**Admin Portal → Config → Service Categories → Offline form** row action), Preventia360-branded, with a ruled blank per field for handwriting and a sign-off block. That's an admin-authenticated feature (uses the admin JWT, not a bearer key) — it isn't reachable from the external data API and isn't meant for third-party integrations; it's listed here only so it isn't confused with the integrator-facing gig-forms endpoint (GET /external/v1/gig-forms).

```
GET /admin/health/gig-services/{serviceCategoryId}/offline-form.
pdf?role=SAHAYAK&mode=preview|download
Authorization: Bearer <admin JWT>
```

Returns `application/pdf` bytes, not the JSON envelope. `mode=preview` (default) sets `Content-Disposition: inline`; `mode=download` sets attachment.